

Storing Images and BLOB files in SQL Server Part 4

By Don Schlichting

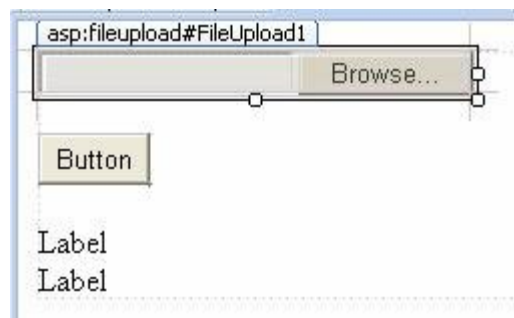
Introduction

BLOBs files are binary data, in other words, not text. Files like an MP3, exe, or pictures are all examples of BLOBs (Binary Large Objects). Often, these types of files may be part of a larger database project. The question usually arises as to how and where to store them, on the File System as files, or inside SQL Server. There are legitimate reasons to justify both. As a very general guide, if the files are small, store them inside SQL server. Another good reason to store binary data inside SQL Server is to obtain the benefit of transactional control. For example, if you need to be sure that an image has been updated or deleted before some other processing step begins, then SQL Server has that control built in. On the other hand, large files, or streaming video, will perform better being served from the Windows file system rather than inside SQL Server. Also, the file system will handle fragmentation better than SQL Server.

This series began by Storing BLOBs inside SQL Server. Part 1 introduced BLOBs and the VARCHAR data type family as well as simple methods for inserting BLOBs into a SQL Server VARCHAR(max) column. Part 2 in this series expanded on this and introduced a Dot Net Binary Write method for displaying those images. In Part 3, an ASPX page was created to accept an image from the web, and store it directly inside SQL Server. This article will focus on storing BLOBs on the Windows File System and using Microsoft SQL Server to organize them from a web page.

File System Example

To begin, we'll build on a control used in the previous examples, the File Upload control. This control will live on a web page and will gather file information, such as file name and directory path to a test image file, and eventually this information will be passed to SQL Server. To begin, create a new page called FileSystemIn.aspx with code behind and drag four controls on to it, a File Upload, a button, and two labels as shown below.



We'll use the default control names for ease of explanations.

```
<asp:FileUpload ID="FileUpload1" runat="server" />
<br />
<br />
<asp:Button ID="Button1" runat="server" onclick="Button1_Click" Text="Button" />
<br />
```

```

<br />
<asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
<br />
<asp:Label ID="Label2" runat="server" Text="Label"></asp:Label>

```

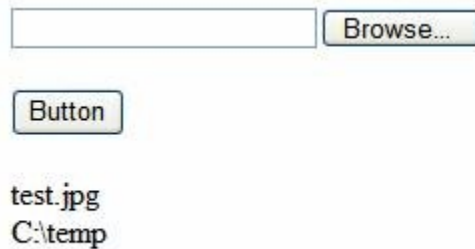
On the code behind page, we'll gather the file name and directory path and display them in the labels. Create a Click event and insert the following code:

```

protected void Button1_Click(object sender, EventArgs e)
{
    Label1.Text = System.IO.Path.GetFileName(FileUpload1.PostedFile.FileName.ToString());
    Label2.Text =
        System.IO.Path.GetDirectoryName(FileUpload1.PostedFile.FileName.ToString());
}

```

When the button is clicked, the "PostedFile", (the file browsed to by the File Upload control) will be displayed.



We can also pass this data to SQL Server through a stored procedure so the database can become our image organizer. First, create a test database to hold the file information from the TSQL below:

```

USE master;
GO
CREATE DATABASE BLOBTest4;
GO
USE BLOBTest4;
GO
CREATE TABLE FileInfo
(
    TheName varchar(50),
    DirPath varchar(50)
);

```

Now we'll create a stored procedure the web page will use to INSERT:

```

CREATE PROCEDURE FileSystemIn
(
    @TheName varchar(50),
    @DirPath varchar(50)
)

```

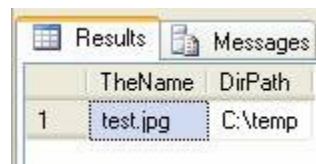
```
AS
INSERT INTO FileInfo
(TheName, DirPath)
VALUES
(@TheName, @DirPath);
```

Next, we'll replace the web page code behind On Click code with a stored procedure. So now, the file information will be saved inside SQL Server rather than displayed on a label.

```
string sTheName, sDirPath;
sTheName = System.IO.Path.GetFileName(FileUpload1.PostedFile.FileName.ToString());
sDirPath = System.IO.Path.GetDirectoryName(FileUpload1.PostedFile.FileName.ToString());

string sConn = @"server=.; database=BLOBTest4; Integrated Security=True";
SqlConnection objConn = new SqlConnection(sConn);
objConn.Open();
SqlCommand objCmd = new SqlCommand("FileSystemIn", objConn);
objCmd.CommandType = CommandType.StoredProcedure;
SqlParameter pTheName = objCmd.Parameters.Add("@TheName", SqlDbType.VarChar,
50);
pTheName.Direction = ParameterDirection.Input;
pTheName.Value = sTheName;
SqlParameter pDirPath = objCmd.Parameters.Add("DirPath", SqlDbType.VarChar, 50);
pDirPath.Direction = ParameterDirection.Input;
pDirPath.Value = sDirPath;
objCmd.ExecuteNonQuery();
objConn.Close();
```

Test the web page, SELECT on the FileInfo table; one row will be returned as shown below.



	TheName	DirPath
1	test.jpg	C:\temp

For an explanation of the On Click Stored Procedure code, please review article 3 in this series. The same SQL Connection and Command objects were used there as well. From here, a web page can be created to view the images by getting the path information from SQL Server.

SQL Server FILESTREAM

There are a couple of problems with the type of implementation demonstrated. First, if files are deleted or added by some method other than the web application, SQL Server will be unaware, and thus out of sync with the file system. The second problem is nightly backup. Now you'll need to backup the directory paths where the files are stored as well as backing up the SQL Server database. In addition, security to these files is now outside of SQL Server controls.

A way to overcome these problems in SQL Server 2008 is to use the new FILESTREAM option for a VARBINARY(MAX) column. FILESTREAMS are physically stored on WINDOWS

NTFS, just like a standard MDF or LDF SQL Server files, but they are specifically created for storing binary data. The `CRATEDATABASE` statement is used to create a special `FILEGROUP` and mark it as a stream. Once the database is created, a column inside a table can be designated as a type `"VARBINARY(MAX) FILESTREAM"`. BLOBs stored inside the `FILESTREAM` are not accessible from the file system. You can't open Windows File explorer and have access to them, meaning security is handled by SQL Server. In addition, the images can be manipulated with standard `INSERT`, `UPDATE`, and `DELETE` statements. So for large BLOBs, or BLOBs with high disk activity like streaming video, SQL Server now has a viable option for handling this type of data.

Conclusion

SQL Server has several options for managing BLOBs or binary data. `VARBINARY (MAX)` in SQL 2005, `IMAGE` data types in older versions, and the new `FILESTREAM` option in SQL Server 2008. Working with them requires a little additional effort compared to standard data types, but the methods and objects used are straightforward to use. Remember that error checking wasn't included in these examples, so make sure to check the `FileUpload` for a data before sending it to SQL Server (in case the end user clicks the button before selecting a file).